

StudyFlow

Sistema Inteligente de Organização de Estudos
v1.0 — StudyFlow

Descrição do Sistema

StudyFlow é uma plataforma web inteligente para organização da rotina acadêmica de estudantes. O sistema centraliza o planejamento de matérias, gerenciamento de tarefas, controle de metas e acompanhamento do progresso semanal, com auxílio de Inteligência Artificial heurística para otimizar a distribuição da carga de estudos.

Tecnologias Utilizadas

Camada	Tecnologia	Versão
Backend	Python + FastAPI	3.11 / 0.100+
ORM / Banco	SQLAlchemy + SQLite	2.x / 3.x
Autenticação	JWT (python-jose) + bcrypt (passlib)	—
Frontend	React + Vite	18.x / 4.x
Roteamento	React Router DOM	6.x
HTTP Client	Axios	1.x
Estilização	Vanilla CSS (Glassmorphism)	—

Arquitetura

O sistema adota arquitetura cliente-servidor desacoplada (SPA + REST API):

- **Frontend (React + Vite):** SPA com roteamento client-side, interceptor Axios injetando JWT automaticamente, UI Glassmorphism responsiva.
- **Backend (FastAPI):** API RESTful modular com routers separados (users, subjects, tasks), middleware JWT, validação Pydantic e módulo de IA heurística.
- **Banco de Dados (SQLite):** Arquivo studyflow.db gerenciado pelo SQLAlchemy ORM, estrutura preparada para migração a PostgreSQL.

Estrutura de Pastas

```
StudyFlow/ ■■■■ backend/ ■ ■■■■ main.py # Ponto de entrada FastAPI ■ ■■■■ database.py #
Configuração SQLAlchemy ■ ■■■■ models.py # Modelos ORM (User, Subject, Task) ■ ■■■■
schemas.py # Schemas Pydantic ■ ■■■■ auth.py # JWT + bcrypt ■ ■■■■ studyflow.db # Banco de
dados SQLite ■ ■■■■ routers/ ■ ■■■■ users.py # POST /users/register, POST /users/login ■
■■■■ subjects.py # CRUD /subjects ■ ■■■■ tasks.py # CRUD /tasks (+ IA heurística) ■■■■
frontend/ ■■■■ src/ ■ ■■■■ App.jsx # Roteamento principal ■ ■■■■ main.jsx # Entry point
React ■ ■■■■ index.css # Design system / Glassmorphism ■ ■■■■ services/api.js # Axios +
interceptor JWT ■ ■■■■ pages/ ■ ■■■■ Login.jsx ■ ■■■■ Register.jsx ■ ■■■■ Dashboard.jsx ■■■■
package.json
```

Instruções de Execução

Pré-requisitos: Python 3.11+, Node.js 18+, npm.

1. Backend:

```
cd backend python -m venv venv venv\Scripts\activate # Windows pip install fastapi uvicorn
sqlalchemy passlib[bcrypt] python-jose[cryptography] pydantic[email] uvicorn main:app
--reload # API disponível em: http://localhost:8000 # Swagger UI em:
http://localhost:8000/docs
```

2. Frontend:

```
cd frontend npm install npm run dev # Interface disponível em: http://localhost:5173
```

3. Atalho (Windows):

Execute run.bat na raiz do projeto para iniciar ambos os servidores automaticamente.

Variáveis de Ambiente

Em produção, substitua as variáveis hardcoded por variáveis de ambiente:

Variável	Descrição
SECRET_KEY	Chave secreta para assinar JWT (mínimo 32 chars)
ACCESS_TOKEN_EXPIRE_MINUTES	TTL do token em minutos (padrão: 1440)
DATABASE_URL	URL do banco (padrão: sqlite:///./studyflow.db)